

Stage 1: Ingestion — Multi-Dialect Voice-to-Structured-JSON

What this layer does:

It takes messy, multi-lingual, or code-switched (e.g., Hinglish) audio clips submitted by citizens, transcribes them into text, and uses an LLM to extract key variables. Instead of manual sorting, it outputs a clean, standardized JSON object.

How to build it:

1. **Audio Ingestion:** Create a simple frontend audio recorder button or an API endpoint that accepts raw audio files (`.mp3`, `.wav`, or `.m4a`).
2. **Transcription (Speech-to-Text):** Send the audio file to an API that excels at handling Indian accents and mixed languages.
 - *Hackathon Choice:* **Groq API** running **Whisper-Large-V3** (incredibly fast and free/cheap) or **OpenAI Whisper API**.
3. **Structured Extraction (Text-to-JSON):** Pass the raw transcript to an LLM with a strict system prompt instructing it to output structured JSON.
 - *Hackathon Choice:* Use **Gemini-2.5-Flash** or **GPT-4o-mini** with **Structured Outputs** (JSON Mode) enabled.

Stage 2: Aggregation — Semantic Text Clustering & Counting

What this layer does:

It prevents duplicates. If 50 people send different voice notes about the "broken water pipeline near Main Market," this layer groups them together into a single **"Master Incident."** It calculates a priority score based on the volume of complaints.

How to build it:

1. **Generate Embeddings:** For every incoming JSON payload, take the `summary` string and turn it into a high-dimensional vector.
 - *Hackathon Choice:* Use the `text-embedding-3-small` model from OpenAI or `all-MiniLM-L6-v2` via the `sentence-transformers` library locally.
2. **Cluster Vector Data:** Store these in a temporary vector array or a lightweight vector database. Use a clustering algorithm to find similar issues.
 - *Hackathon Choice:* **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) from `scikit-learn`. It's perfect because you don't need to tell it how many clusters to expect in advance.

3. **Calculate Priority:** Count the number of items in a cluster.

$$\text{Priority Score} = \text{Count} \times \text{Urgency Multiplier (High=3, Med=2, Low=1)}$$

Stage 3: Action — Automated Official Letter Draft & Router

What this layer does:

Once the MP selects a high-priority "Master Incident" on their dashboard, this layer uses an LLM to compile all the clustered citizen statements into a formal, highly professional official grievance letter addressed to the respective administrative body.

How to build it:

1. **Target Routing:** Map the JSON **category** to the correct government department (e.g., "Water Supply" $\rightarrow$ *The Chief Engineer, Jal Board*).
2. **Context Assembly:** Feed the LLM the aggregated stats: Total citizens complaining, specific location details, and a bulleted list of the individual summaries compiled from Stage 2.
3. **Template Prompting:** Use a highly structured prompt to enforce a formal letter format.
4. **PDF Generation:** Take the markdown text outputted by the LLM and convert it into a downloadable PDF.
 - *Hackathon Choice:* Use **FPDF2** or **ReportLab** in Python to compile the final document.

Tech Stack Recommendation for the Hackathon:

- **Backend:** Python Fast API (highly modular, native JSON support, fast asynchronous endpoints).
- **Frontend:** React (Vite) + TailwindCSS (for building a highly polished, clean 3-step dashboard).
- **Inter-layer Communication:** Simple REST API endpoints (`/upload-audio` $\rightarrow$ `/get-clusters` $\rightarrow$ `/generate-letter`).